

Core Java

Interview Preparation — Deep Dive

10 Topics · Concepts · Code · Q&A

Prepared for intermediate Java developers (1-3 years experience)
Crash-course revision with interview-grade depth

Table of Contents

1. [Java Fundamentals](#) — JVM/JDK/JRE, primitives, pass-by-value
2. [OOP Principles](#) — Encapsulation, Inheritance, Polymorphism, Abstraction
3. [Classes Deep Dive](#) — Constructors, this/super, static, final, inner classes, immutability
4. [Strings](#) — String pool, immutability, StringBuilder vs StringBuffer
5. [Exception Handling](#) — Checked vs unchecked, try-with-resources, custom exceptions
6. [Collections Framework](#) — List/Set/Map/Queue, HashMap & ConcurrentHashMap internals
7. [Generics](#) — Type safety, wildcards, PECS, type erasure
8. [Multithreading & Concurrency](#) — synchronized, volatile, atomics, executors, locks
9. [Java 8 Features](#) — Lambdas, Streams, Optional, Date/Time API
10. [Memory Management & Garbage Collection](#) — Heap structure, generations, GC algorithms

Topic 1 — Java Fundamentals

1.1 JDK vs JRE vs JVM

- **JVM (Java Virtual Machine)** — Runtime engine that executes bytecode. Platform-dependent but makes Java platform-independent. Has Class Loader, Bytecode Verifier, Execution Engine (Interpreter + JIT), Garbage Collector.
- **JRE (Java Runtime Environment)** — JVM + core libraries (rt.jar) + supporting files. What you need to *run* Java apps.
- **JDK (Java Development Kit)** — JRE + development tools (javac, javadoc, jar, debugger). What you need to *develop* Java apps.

Compilation flow

```
.java file → javac → .class (bytecode) → JVM → Machine code
```

Why "Write Once, Run Anywhere": bytecode runs on any JVM, and JVMs exist for every major OS.

1.2 Primitive vs Reference Types

- **Primitives (8):** `byte` (1B), `short` (2B), `int` (4B), `long` (8B), `float` (4B), `double` (8B), `char` (2B), `boolean` — stored on the **Stack**.
- **References:** Objects, arrays — reference on Stack, actual object on Heap.

1.3 Pass by Value

Java is **always pass-by-value**. For objects, the *value of the reference* (a copy of the reference) is passed. So you can modify the object's state but cannot reassign the caller's reference.

```
public class PassByValueDemo {
    public static void main(String[] args) {
        int x = 10;
        modifyPrimitive(x);
        System.out.println(x); // 10 (unchanged)

        StringBuilder sb = new StringBuilder("Hello");
        modifyObject(sb);
        System.out.println(sb); // "Hello World" (state changed)

        reassignObject(sb);
        System.out.println(sb); // "Hello World" (reassignment lost!)
    }
    static void modifyPrimitive(int n) { n = 99; }
    static void modifyObject(StringBuilder s) { s.append(" World"); }
    static void reassignObject(StringBuilder s) { s = new StringBuilder("New"); }
}
```

1.4 Interview Q&A

Q: Is Java pass-by-value or pass-by-reference?

Always pass-by-value. For reference types, the *value of the reference* is copied — not the object itself.

Q: Why is Java platform-independent but JVM is platform-dependent?

Java compiles to platform-neutral bytecode. The JVM translates bytecode to OS-specific machine instructions, so each OS needs its own JVM.

Q: What does JIT do?

Just-In-Time compiler converts *frequently executed* bytecode (hotspots) into native machine code at runtime, caching it for performance.

Q: Difference between `==` and `.equals()`?

`==` compares references for objects, values for primitives. `.equals()` compares logical/content equality (when properly overridden).

Q: Default value of int, boolean, object reference?

For instance/static variables: `int` =0, `boolean` =false, reference= `null` . **Local variables have NO default** — must be initialized before use.

Q: Can you run a .class file without JDK?

Yes — only JRE is needed to run. JDK is required only for compilation.

Q: What is bytecode?

An intermediate, platform-independent instruction set (in `.class` files) executed by the JVM.

Topic 2 — OOP Principles

2.1 Encapsulation

Wrapping data and code together as a single unit, restricting direct access to internal state.

How achieved: private fields + public getters/setters + validation in setters.

```
public class BankAccount {
    private double balance;
    private final String accountNo;

    public BankAccount(String accountNo) {
        this.accountNo = accountNo;
        this.balance = 0.0;
    }
    public double getBalance() { return balance; }
    public void deposit(double amount) {
        if (amount <= 0) throw new IllegalArgumentException("Amount must be positive");
        this.balance += amount;
    }
    public void withdraw(double amount) {
        if (amount > balance) throw new IllegalStateException("Insufficient funds");
        this.balance -= amount;
    }
}
```

Q: Encapsulation vs Abstraction?

Encapsulation = HOW you hide data (private + getters/setters). Abstraction = WHAT you hide (implementation details behind interface/abstract class).

Q: Can encapsulation be broken?

Yes — via Reflection (`field.setAccessible(true)`), serialization, or returning mutable references from getters.

2.2 Inheritance

Mechanism where one class acquires properties/methods of another. Promotes **code reuse** and an **"IS-A"** relationship.

Type	Supported?
Single ($A \rightarrow B$)	✓
Multilevel ($A \rightarrow B \rightarrow C$)	✓
Hierarchical ($A \rightarrow B, A \rightarrow C$)	✓
Multiple via classes	✗ (Diamond Problem)
Multiple via interfaces	✓

Why no multiple class inheritance? Diamond problem — ambiguity if both parents define same method. Interfaces solve this since Java 8 with explicit override (`A.super.method()`).

```
// Diamond problem with interfaces (Java 8+)
interface A { default void hello() { System.out.println("A"); } }
interface B { default void hello() { System.out.println("B"); } }
class C implements A, B {
    @Override
    public void hello() { A.super.hello(); } // explicit pick
}
```

Q: What is super used for?

Call parent constructor (`super(args)`), access parent method (`super.method()`), parent field (`super.field`). Constructor call must be the first statement.

Q: Are constructors inherited?

No. But child constructor must invoke parent (explicit or implicit `super()`).

Q: Can we extend a final class?

No. `final` on class prevents inheritance (e.g., `String`, `Integer`).

Q: IS-A vs HAS-A?

IS-A = inheritance (`Dog extends Animal`). HAS-A = composition (`Car has Engine`). Prefer composition over inheritance.

2.3 Polymorphism

Compile-time (Method Overloading)

- Same method name, different parameter list (type, count, or order)
- Different return type alone is NOT enough
- Resolved at compile time (static binding)

Runtime (Method Overriding)

- Child provides specific implementation of parent's method
- Same signature; return type same or covariant
- Access modifier same or wider (can't reduce visibility)
- `static`, `final`, `private` methods cannot be overridden
- Resolved at runtime (dynamic dispatch)

```
class Shape { public double area() { return 0; } }
class Circle extends Shape {
    double r;
    Circle(double r) { this.r = r; }
    @Override public double area() { return Math.PI * r * r; }
}
class Square extends Shape {
    double s;
    Square(double s) { this.s = s; }
    @Override public double area() { return s * s; }
}

Shape s = new Circle(5);
System.out.println(s.area()); // 78.53 - Circle's area() called at runtime!
s = new Square(4);
System.out.println(s.area()); // 16.0
```

Q: Can we override a static method?

No. Static methods belong to class — they can be **hidden** (redeclared) but resolved at compile time, not runtime. Not polymorphic.

Q: Can we override a private method?

No. Private methods aren't visible to subclass; declaring same-named method in child is a new method.

Q: Can the main method be overloaded?

Yes — multiple `main` methods allowed. JVM only invokes `public static void main(String[] args)`.

Q: What is covariant return type?

Overriding method can return a subtype of parent's return type (since Java 5).

Feature	Overloading	Overriding
Location	Same class	Parent + Child
Method signature	Different	Same
Binding	Compile-time	Runtime
Return type	Can differ	Same or covariant

2.4 Abstraction

Hiding implementation details, exposing only essential features. Achieved via **abstract classes** and **interfaces**.

Feature	Abstract Class	Interface
Methods	Abstract + concrete	Abstract + default + static + private (Java 8/9+)
Variables	Any (instance, static, final)	<code>public static final</code> only
Constructor	✓ Yes	✗ No
Multiple inheritance	✗ Single	✓ Multiple
State	✓ Can hold state	✗ Constants only
Keyword	<code>extends</code>	<code>implements</code>

Rule of thumb: Use abstract class when classes share code/state. Use interface for capability contracts (`Comparable` , `Runnable` , `Serializable`).

```

abstract class Vehicle {
    protected String brand;
    public Vehicle(String brand) { this.brand = brand; }
    public void start() { System.out.println(brand + " starting..."); }
    public abstract void fuelType();
}

interface Payment {
    void pay(double amount);
    default void receipt() { System.out.println("Thank you"); }
    static boolean isValid(double amt) { return amt > 0; }
}

```

Q: Can an abstract class have a constructor?

Yes. Even though you can't instantiate it, the constructor runs when a subclass is instantiated (via `super()`).

Q: Why default methods in Java 8?

To **evolve interfaces** without breaking existing implementations. Example: `Collection` got `stream()` as default.

Q: What is a marker interface?

Interface with no methods — used to "mark" a class for special treatment. Examples: `Serializable` , `Cloneable` , `Remote` .

Q: What is a functional interface?

Interface with exactly ONE abstract method (SAM). Annotated `@FunctionalInterface` . Used for lambdas. Examples: `Runnable` , `Comparator` , `Function` .

Q: Is Java 100% object-oriented?

No — uses primitives (int, char, etc.) which aren't objects. ~90% OOP.

Topic 3 — Classes Deep Dive

3.1 Constructors

Special method that initializes an object. Same name as class, no return type, called automatically with `new`.

Types

- **Default (no-arg)** — Provided by compiler if you don't define any
- **Parameterized** — Takes arguments
- **Copy constructor** — Java doesn't auto-provide; write manually
- **Private** — Used in Singleton, Builder, factory methods

```
public class Employee {
    private String name; private int id;

    public Employee() { this("Unknown", 0); }
    public Employee(String name, int id) { this.name = name; this.id = id; }
    public Employee(String name) { this(name, 0); } // chaining via this()
    public Employee(Employee other) { this(other.name, other.id); } // copy
}
```

Q: Can a constructor be private?

Yes. Used in Singleton, utility classes (Math, Collections), static factory methods like `Integer.valueOf()`.

Q: Can a constructor call another constructor?

Yes — `this(args)` for same-class chaining, `super(args)` for parent. Must be the FIRST statement.

Q: Can a constructor be inherited?

No. But child constructor implicitly invokes parent's via `super()`.

Q: Can a constructor throw exceptions?

Yes — both checked and unchecked. If it throws, the object is never created.

Q: Can a constructor be final, static, abstract, synchronized?

No to all. final = constructors aren't inherited; static = needs `this`; abstract = must have a body; synchronized = no two threads can construct same object.

Q: Order of execution during object creation?

1) Static blocks of parent (once per class load), 2) Static blocks of child, 3) Parent instance fields + constructor, 4) Child instance fields + constructor.

3.2 this and super

Use	this	super
Reference	Current object	Parent class
Field	<code>this.field</code>	<code>super.field</code>
Method	<code>this.method()</code>	<code>super.method()</code>
Constructor	<code>this(args)</code> — first line	<code>super(args)</code> — first line

Q: Can both this() and super() be in same constructor?

No — each must be the first statement.

3.3 static Keyword

`static` = belongs to class, not instance. One copy shared across objects.

- **Static variable** — class-level, shared across instances
- **Static method** — called without object; cannot access non-static directly
- **Static block** — runs once when class is loaded
- **Static nested class** — doesn't need outer instance

```
public class Counter {
    private static int count = 0;
    static { System.out.println("Class loaded"); }
    public Counter() { count++; }
    public static int getCount() { return count; }
}
```

Q: Can we override static methods?

No — that's **method hiding**, resolved at compile time. Not polymorphic.

Q: Why is main static?

JVM calls main without instantiating the class. If non-static, JVM wouldn't know which constructor to call.

Q: Can a class be static?

Only nested classes can be static. Top-level classes cannot.

3.4 final Keyword

Where	Meaning
final variable	Cannot be reassigned (must be initialized once)
final method	Cannot be overridden
final class	Cannot be extended (e.g., String, Integer)

Final reference variable: reference is final, but referenced object's state may still be mutated.

```
final List<String> list = new ArrayList<>();
list.add("A"); // ✓ allowed
list = new ArrayList<>(); // ✗ reassignment forbidden
```

Q: Difference between final, finally, finalize()?

final = keyword (immutability). **finally** = block (always executes after try/catch). **finalize()** = method called by GC (deprecated since Java 9).

3.5 Access Modifiers

Modifier	Same Class	Same Pkg	Subclass (other pkg)	Everywhere
private	✓	✗	✗	✗
default	✓	✓	✗	✗
protected	✓	✓	✓	✗
public	✓	✓	✓	✓

Q: Can a top-level class be private?

No. Only nested classes can be private.

Q: Can an overriding method reduce visibility?

No — must be same or wider. Violates Liskov Substitution Principle otherwise.

3.6 Inner / Nested Classes

- **Static nested class** — no outer instance needed
- **Inner class** (non-static) — needs outer instance: `outer.new Inner()`
- **Local inner class** — defined inside a method
- **Anonymous inner class** — no name, on-the-fly extends/implements

```
// Anonymous
Runnable r = new Runnable() {
    @Override public void run() { System.out.println("Running"); }
};
```

3.7 Object Class & equals/hashCode Contract

Every Java class implicitly extends `Object`. Key methods: `equals`, `hashCode`, `toString`, `getClass`, `clone`, `wait/notify/notifyAll`.

Contract

1. If `a.equals(b)` → `a.hashCode() == b.hashCode()` (mandatory)
2. Same hashCode does NOT require equals (collisions allowed)
3. equals must be: reflexive, symmetric, transitive, consistent, null-safe

```
public class Student {
    private String id; private String name;
    @Override public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Student)) return false;
        return Objects.equals(id, ((Student) o).id);
    }
    @Override public int hashCode() { return Objects.hash(id); }
}
```

Trap: Override equals without hashCode → equal objects with different hashCodes break HashMap/HashSet. **Always override both together.**

3.8 Immutable Classes

State cannot change after creation. Examples: `String`, `Integer`, `LocalDate`.

Rules to design immutable class

1. Declare class as `final`
2. All fields `private final`
3. No setters
4. Initialize via constructor only
5. Defensive copies for mutable fields (in constructor and getter)

```
public final class ImmutableEmployee {
    private final String name;
    private final List<String> roles;
    public ImmutableEmployee(String name, List<String> roles) {
        this.name = name;
        this.roles = new ArrayList<>(roles); // defensive copy
    }
    public String getName() { return name; }
    public List<String> getRoles() { return new ArrayList<>(roles); }
}
```

Q: Why is String immutable?

String pool caching, thread safety, security (class loader paths, DB URLs), hashCode caching for HashMap, class loader integrity.

Topic 4 — Strings

4.1 Basics

String is an **object** (not primitive) and **immutable**.

- Java 8 and earlier: `private final char[] value`
- Java 9+: `private final byte[] value` + `coder` flag (Compact Strings — saves memory for ASCII)

```
String s1 = "Hello";           // String literal (pool)
String s2 = new String("Hello"); // Heap object + literal in pool
```

4.2 String Pool (SCP)

A special memory region inside Heap (since Java 7; before that, in PermGen). Stores unique String literals to enable reuse and save memory.

```
String a = "Java";
String b = "Java";
String c = new String("Java");

System.out.println(a == b); // true  (same pool reference)
System.out.println(a == c); // false (different objects)
System.out.println(a.equals(c)); // true
```

intern()

Forces a String into the pool. If pool already has equal string, returns that reference.

```
String c = new String("Java");
String d = c.intern();
String a = "Java";
System.out.println(d == a); // true
```

4.3 Why is String Immutable? (Top question)

1. **String Pool optimization** — sharing pool entries safely
2. **Thread safety** — readable concurrently without locks
3. **Security** — file paths, DB URLs, class loader paths can't be tampered with
4. **HashCode caching** — computed once, reused (vital for HashMap keys)
5. **Class loader integrity** — JVM uses Strings as class names

4.4 String vs StringBuilder vs StringBuffer

Feature	String	StringBuilder	StringBuffer
Mutability	Immutable	Mutable	Mutable
Thread-safe	Yes (immutable)	✗ No	✓ Yes (synchronized)
Performance	Slow for concat	Fastest	Slower than SB
Introduced	1.0	Java 5	1.0
Use case	Few changes	Single-thread mutations	Multi-thread mutations

Classic trap: `String s = ""; for (int i=0; i<10000; i++) s += i;` → creates 10000 new String objects ($O(n^2)$). Use `StringBuilder` instead.

```
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 10000; i++) sb.append(i);
String result = sb.toString();
```

StringBuilder internals

- Default capacity = 16
- When full, new array of `oldCap * 2 + 2`
- Pre-size for performance: `new StringBuilder(1024)`

4.5 Common String Methods

```
s.length(); s.charAt(0); s.indexOf("x"); s.substring(0, 5);
s.toUpperCase(); s.toLowerCase(); s.trim(); s.strip(); // strip Java 11+
s.replace('l', 'L'); s.replaceAll("l+", "L"); s.split(" ");
s.contains("x"); s.startsWith("a"); s.endsWith("z");
s.equals(t); s.equalsIgnoreCase(t); s.compareTo(t);
s.isEmpty(); s.isBlank(); // isBlank Java 11+
s.concat("!"); s.toCharArray();
String.valueOf(123); String.format("Hi %s", name);
String.join(",", "a", "b", "c");
".".repeat(5); // Java 11+
```

4.6 == vs equals() vs compareTo()

- `==` — reference comparison
- `equals()` — content comparison (case-sensitive); use `equalsIgnoreCase()` for case-insensitive
- `compareTo()` — lexicographic, returns 0/+/- int

Null-safe comparison: `"expected".equals(input)` works even if `input` is null. Use `Objects.equals(a, b)` for both-side null safety.

4.7 Concatenation Internals

For string literals (compile-time constants): compiler concatenates at compile time. For variables: compiler translates `+` into `StringBuilder` calls. Java 9+ uses `invokedynamic` + `StringConcatFactory` — more efficient.

4.8 Common Coding Problems

Reverse a String

```
String reversed = new StringBuilder("hello").reverse().toString();
```

Check Palindrome

```
boolean isPalindrome(String s) {
    int i = 0, j = s.length() - 1;
    while (i < j) if (s.charAt(i++) != s.charAt(j--)) return false;
    return true;
}
```

Character Frequency

```
Map<Character, Integer> count = new HashMap<>();
for (char c : "programming".toCharArray()) count.merge(c, 1, Integer::sum);
```

Check Anagram

```
boolean isAnagram(String a, String b) {
    if (a.length() != b.length()) return false;
    char[] ca = a.toCharArray(); char[] cb = b.toCharArray();
    Arrays.sort(ca); Arrays.sort(cb);
    return Arrays.equals(ca, cb);
}
```

First Non-Repeating Character

```
char firstUnique(String s) {
    Map<Character, Integer> count = new LinkedHashMap<>();
    for (char c : s.toCharArray()) count.merge(c, 1, Integer::sum);
    for (var e : count.entrySet()) if (e.getValue() == 1) return e.getKey();
    return '_';
}
```

4.9 Top Q&A

Q: How many objects are created here?

`String s1 = "abc"; String s2 = "abc"; String s3 = new String("abc"); String s4 = new String("abc");` → **3 objects**: 1 in pool (shared by s1, s2), 2 separate heap objects.

Q: Where is the String Pool stored?

Java 6: PermGen. Java 7+: Heap (so it's eligible for GC). Java 8+: PermGen replaced by Metaspace, but pool is still in heap.

Q: Why are Strings used as HashMap keys often?

Immutable → hashCode never changes → safe and efficient. Plus hashCode is cached.

Q: replace() vs replaceAll()?

`replace` = literal replacement. `replaceAll` = regex-based. `"a.b".replace(".", "-")` → `"a-b"`.
`"a.b".replaceAll(".", "-")` → `"---"` (regex `.` matches any char).

Topic 5 — Exception Handling

5.1 Exception Hierarchy

```
Throwable
├── Error (OutOfMemoryError, StackOverflowError) – JVM-level, NOT to be caught
└── Exception
    ├── Checked (IOException, SQLException, ClassNotFoundException, InterruptedException)
    └── RuntimeException (NPE, ArithmeticException, ClassCastException, IllegalArgumentException, ...)
```

5.2 Checked vs Unchecked Exceptions

Aspect	Checked	Unchecked
Parent	Exception (excl. RuntimeException)	RuntimeException
Compile-time check	✓ Compiler enforces	✗ Not enforced
Must declare?	✓ via throws or try-catch	✗ Optional
Represents	External / recoverable	Programming bugs
Examples	IOException, SQLException, ClassNotFoundException, InterruptedException, FileNotFoundException	NullPointerException, ArithmeticException, ArrayIndexOutOfBoundsException, IllegalArgumentException, ClassCastException, NumberFormatException

Q: Is IOException checked or unchecked?

Checked. Direct subclass of Exception. Methods that throw it must declare or catch.

Q: Base class for unchecked exceptions?

`RuntimeException`.

Q: Name 5 unchecked exceptions.

`NullPointerException`, `ArithmeticException`, `ArrayIndexOutOfBoundsException`, `ClassCastException`, `IllegalArgumentException`, `NumberFormatException`, `IllegalStateException`, `ConcurrentModificationException`.

Q: Name 5 checked exceptions.

`IOException`, `FileNotFoundException`, `SQLException`, `ClassNotFoundException`, `InterruptedException`, `CloneNotSupportedException`.

5.3 try-catch-finally

Rules

- Order matters — child exception caught before parent
- `finally` always runs (even if try/catch throws)
- Java 7+ multi-catch: `catch (IOException | SQLException e)` — variable `e` implicitly final
- A try must have at least one catch OR finally

When does finally NOT execute?

1. `System.exit()` called
2. JVM crashes (kill -9)
3. Thread killed forcibly
4. Infinite loop in try/catch

```
try {
    risky();
} catch (FileNotFoundException e) {
    log.error("File missing", e);
} catch (IOException e) {
    log.error("Read failed", e);
} finally {
    cleanup();
}
```

Trap: If both try and finally throw, the finally exception **suppresses** the original. The try exception is lost (unless try-with-resources is used).

Q: Can return be in finally?

Yes — but it overrides any return from try/catch. Bad practice.

5.4 throw vs throws

Keyword	throw	throws
Purpose	Actually throws an exception	Declares method may throw
Where	Inside method body	After method signature
Followed by	Exception object	Exception class names
Mandatory?	Optional	Required for checked

5.5 try-with-resources (Java 7+)

Auto-closes resources implementing `AutoCloseable`.

```
try (BufferedReader br = new BufferedReader(new FileReader("f.txt"));
    PrintWriter pw = new PrintWriter("out.txt")) {
    // use br and pw – closed in REVERSE order automatically
} catch (IOException e) { e.printStackTrace(); }
```

Preserves both exceptions via **suppressed exceptions** (`e.getSuppressed()`).

5.6 Custom Exceptions

```
public class UserNotFoundException extends RuntimeException {
    public UserNotFoundException(String userId) {
        super("User not found: " + userId);
    }
}

public class ServiceException extends RuntimeException {
    public ServiceException(String msg, Throwable cause) { super(msg, cause); }
}
```

5.7 Exception Chaining (Wrapping)

```
try {
    db.fetch();
} catch (SQLException e) {
    throw new ServiceException("Failed to fetch user", e); // wrap, preserve cause
}
// Later: e.getCause() returns original SQLException
```

5.8 Real-World Scenario (DB + External API)

```
try {
    repo.save(data);
    external.call();
} catch (DataAccessException e) {
    log.error("DB save failed", e);
    throw new ServiceException("Persistence error", e);
} catch (RestClientException e) {
    log.error("External call failed", e);
    throw new ServiceException("Upstream service error", e);
} catch (Exception e) { // catch-all safety net
    log.error("Unexpected error", e);
    throw new ServiceException("Operation failed", e);
}
```

If DB fetch fails: Spring/JPA → `DataAccessException` (parent), or specific subclasses like `EmptyResultDataAccessException`, `DataIntegrityViolationException`, `QueryTimeoutException`. Raw JDBC →

`SQLException`.

Common catch-all: `Exception` (don't use `Throwable` — catches Errors too).

5.9 Best Practices

DO

- Catch specific first, generic last
- Always log with stack trace
- Wrap exceptions when crossing layers — preserve cause
- Use try-with-resources
- Throw early (fail-fast), catch late (at appropriate boundary)

DON'T

- Don't swallow exceptions (empty catch blocks hide bugs)
- Don't catch `Throwable` (catches `Error` too)
- Don't use exceptions for flow control (expensive)
- Don't ignore `InterruptedException` — re-interrupt or propagate

```
// Proper InterruptedException handling
catch (InterruptedException e) {
    Thread.currentThread().interrupt(); // restore flag
    throw new MyException("interrupted", e);
}
```

5.10 Top Q&A

Q: Difference between final, finally, finalize()?

`final` = immutability/non-overridability. `finally` = block that always executes. `finalize()` = pre-GC cleanup (deprecated Java 9, removed Java 18).

Q: Difference between exception and error?

Exception = app-level, recoverable, should be caught. Error = JVM/system-level, unrecoverable, should NOT be caught.

Q: ClassNotFoundException vs NoClassDefFoundError?

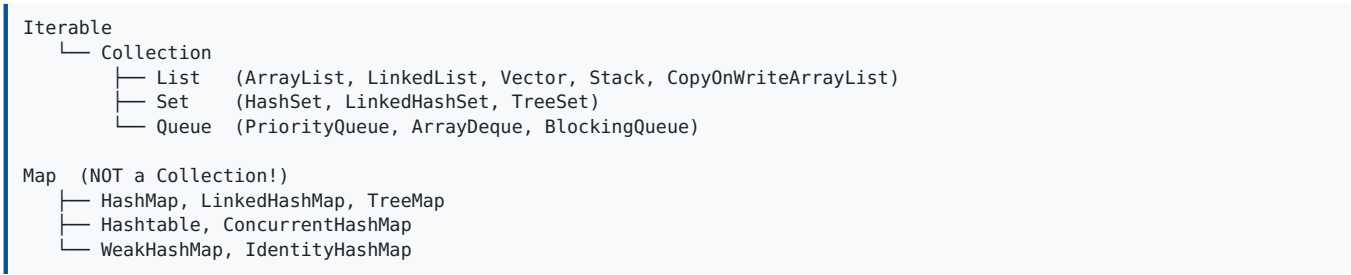
Former: checked, lookup failed at runtime via `Class.forName()`. Latter: Error, class was present at compile time but missing at runtime (jar removed, classpath issue).

Q: ConcurrentModificationException?

Thrown when a non-concurrent collection is modified while being iterated (fail-fast). Fix: use `Iterator.remove()` or `CopyOnWriteArrayList`.

Topic 6 — Collections Framework

6.1 Hierarchy



6.2 List Interface

Implementation	Backing	Thread-safe	Random Access
ArrayList	Dynamic array	✗	O(1)
LinkedList	Doubly linked list	✗	O(n)
Vector	Dynamic array	✓ legacy	O(1)
Stack	Extends Vector	✓	—
CopyOnWriteArrayList	Array, copy-on-write	✓	O(1)

ArrayList Internals

- Backed by `Object[] elementData`
- Default capacity = 10
- Grows by `oldCap * 1.5` when full → `newCap = oldCap + (oldCap >> 1)`
- `add(E)` = amortized O(1); `add(index, E)` = O(n); `get(index)` = O(1)

ArrayList vs LinkedList

Operation	ArrayList	LinkedList
get(index)	O(1)	O(n)
add(E) (end)	O(1) amortized	O(1)
add(0, E) (front)	O(n)	O(1)
Memory	Compact	High overhead per node
Cache locality	Excellent	Poor

In real workloads, **ArrayList is almost always faster**. LinkedList's O(1) insertion is overshadowed by traversal cost + cache misses.

6.3 Set Interface

Implementation	Order	Null	Notes
HashSet	None	1 null	Internally a HashMap with dummy values
LinkedHashSet	Insertion	1 null	HashSet + linked list
TreeSet	Sorted	✗ no null	Red-Black tree, O(log n)

```
TreeSet<Integer> ts = new TreeSet<>();
ts.add(5); ts.add(1); ts.add(3);
ts.first();      // 1
ts.last();       // 5
ts.floor(4);     // 3 (greatest <= 4)
ts.ceiling(4);   // 5 (smallest >= 4)
ts.headSet(3);   // [1]
ts.tailSet(3);   // [3, 5]
```

6.4 HashMap Deep Dive (Critical!)

Internal structure

```
transient Node<K,V>[] table;           // bucket array
static class Node<K,V> {
    final int hash; final K key; V value;
    Node<K,V> next;                    // linked list for collisions
}
```

Default constants

Constant	Value
DEFAULT_INITIAL_CAPACITY	16
DEFAULT_LOAD_FACTOR	0.75
TREEIFY_THRESHOLD	8 (list → tree)
UNTREEIFY_THRESHOLD	6 (tree → list)
MIN_TREEIFY_CAPACITY	64

put(key, value) flow

1. Compute hash: `hash = key.hashCode() ^ (hashCode >>> 16)`
2. Bucket index: `(n - 1) & hash` (n = table length)
3. If bucket empty → place node
4. If bucket has entries → traverse list/tree; if key matches, replace; else append
5. If list size ≥ 8 AND table ≥ 64 → **treeify** (Red-Black tree)
6. If size > threshold (capacity × loadFactor) → **resize** (double capacity, rehash)

Why capacity is a power of 2

So `(n - 1) & hash` works as a fast modulo. `&` is much faster than `%`. Even if you pass `new HashMap<>(13)`, internally rounded up to 16.

Why load factor 0.75

Trade-off between memory (lower → more space, fewer collisions) and time (higher → more collisions, less space).

Q: Time complexity of HashMap operations?

Average O(1). Worst case O(log n) since Java 8 (treeification); O(n) before Java 8.

Q: What if two keys have same hashCode but aren't equal?

Hash collision. Both go to same bucket as separate nodes. `equals()` distinguishes them.

Q: What if equals() and hashCode() contract is broken?

Two equal keys with different hashCodes → go to different buckets → second put adds rather than replaces → broken behavior. **Always override both together.**

6.5 ConcurrentHashMap Deep Dive

Java 7 — Segment-based locking

Map divided into 16 segments, each its own mini-HashMap with its own lock. Up to 16 concurrent writes possible.

Java 8+ — Node-level locking + CAS

- Reads → **completely lock-free** (volatile reads)
- Writes to empty bucket → **CAS** (Compare-And-Swap)
- Writes to non-empty bucket → `synchronized` on first node only
- Treeification just like HashMap

HashMap vs ConcurrentHashMap

Feature	HashMap	ConcurrentHashMap
Thread-safe	✗	✓
Null keys/values	1 null key, multi null values	✗ No nulls (NPE)
Locking	None	Bucket-level (Java 8+)
Iterator	Fail-fast (CME)	Fail-safe / weakly consistent
Introduced	Java 1.2	Java 1.5

Why no nulls in ConcurrentHashMap?

If `map.get(key)` returns null, you can't tell if the key is absent or value is genuinely null — ambiguous in race conditions.

Atomic compound operations

```
map.putIfAbsent("a", 1);           // atomic
map.computeIfAbsent("a", k -> 1);  // atomic
map.computeIfPresent("a", (k, v) -> v + 1); // atomic
map.compute("a", (k, v) -> (v == null) ? 1 : v + 1); // atomic
map.merge("a", 1, Integer::sum);    // atomic – perfect for counters
```

Trap: Without these, check-then-act is racy:

```
if (!map.containsKey("a")) { map.put("a", 1); } // ✗ NOT atomic
```

6.6 LinkedHashMap & LRU Cache

HashMap + doubly-linked list → maintains insertion order (or access order for LRU mode).

```
class LRUCache<K, V> extends LinkedHashMap<K, V> {
    private final int capacity;
    public LRUCache(int capacity) {
        super(capacity, 0.75f, true); // accessOrder = true
        this.capacity = capacity;
    }
    @Override
    protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
        return size() > capacity;
    }
}
```

6.7 Queue & Deque

Action	Throws Exception	Returns Special Value
Insert	<code>add(e)</code>	<code>offer(e)</code>
Remove	<code>remove()</code>	<code>poll()</code>
Examine	<code>element()</code>	<code>peek()</code>

```
// PriorityQueue (min-heap by default)
PriorityQueue<Integer> pq = new PriorityQueue<>();
PriorityQueue<Integer> max = new PriorityQueue<>(Comparator.reverseOrder());

// ArrayDeque – modern stack/queue (preferred over Stack class)
Deque<Integer> stack = new ArrayDeque<>();
stack.push(1); stack.push(2); stack.push(3);
stack.pop(); // 3
```

6.8 Iterator vs ListIterator vs Spliterator

- **Iterator** — forward only; `hasNext()`, `next()`, `remove()`
- **ListIterator** — bidirectional; only for List
- **Spliterator** (Java 8+) — for parallel iteration (used by Streams)

6.9 Fail-fast vs Fail-safe

Aspect	Fail-fast	Fail-safe
Behavior on modification	Throws <code>ConcurrentModificationException</code>	Doesn't throw
Mechanism	Tracks <code>modCount</code>	Iterates over snapshot/copy
Examples	<code>ArrayList</code> , <code>HashMap</code> , <code>HashSet</code>	<code>CopyOnWriteArrayList</code> , <code>ConcurrentHashMap</code>

```
// Fail-fast example
List<Integer> list = new ArrayList<>(List.of(1, 2, 3));
for (int n : list) {
    if (n == 2) list.remove(Integer.valueOf(2)); // CME!
}

// Safe alternative
Iterator<Integer> it = list.iterator();
while (it.hasNext()) { if (it.next() == 2) it.remove(); } // ✓
```

6.10 Comparable vs Comparator

Feature	Comparable	Comparator
Package	<code>java.lang</code>	<code>java.util</code>
Method	<code>int compareTo(T o)</code>	<code>int compare(T o1, T o2)</code>
Sorts on	"Natural" order (one)	Any custom order (many)
Modifies class?	Yes	No (external)

```
// Comparator chaining (Java 8+)
Comparator<Employee> cmp = Comparator.comparing(Employee::getName)
    .thenComparingInt(Employee::getId)
    .reversed();
employees.sort(cmp);
employees.sort(Comparator.nullsFirst(Comparator.comparing(Employee::getName)));
```

Pitfall: Don't use `a.id - b.id` for `int` — can overflow for large negatives. Use `Integer.compare(a.id, b.id)`.

6.11 Top Q&A

Q: Difference between Collection and Collections?

`Collection` = root interface. `Collections` = utility class with static helpers (`sort`, `unmodifiableXxx`, `synchronizedXxx`).

Q: Why is Map not part of Collection hierarchy?

`Map` stores key-value pairs, not single elements. `Collection`'s contract (e.g., `add(E)`) doesn't fit.

Q: ArrayList vs Vector?

`ArrayList`: not synchronized, faster, modern. `Vector`: synchronized, legacy (Java 1.0), slower.

Q: How to make a collection thread-safe?

Use `Collections.synchronizedXxx()`, or concurrent classes (`ConcurrentHashMap`, `CopyOnWriteArrayList`), or explicit locks.

Q: Arrays.asList() trap?

Returns a **fixed-size** list backed by the array. `add()` / `remove()` throws `UnsupportedOperationException`. Wrap in `new ArrayList<>(...)` for mutability.

Topic 7 — Generics

7.1 Why Generics?

Compile-time type safety, no casts, code reusability. Pre-Java 5: raw types caused `ClassCastException` at runtime.

```
List<String> list = new ArrayList<>();
list.add("hello");
list.add(42);           // ✕ compile error
String s = list.get(0); // no cast needed
```

7.2 Generic Classes & Methods

```
public class Box<T> {
    private T content;
    public void set(T content) { this.content = content; }
    public T get() { return content; }
}

public class Pair<K, V> {
    private K key; private V value;
    public Pair(K k, V v) { this.key = k; this.value = v; }
}

// Generic method — <T> before return type
public static <T> T pickFirst(List<T> list) {
    return list.get(0);
}
```

Naming conventions

T = Type, E = Element, K/V = Key/Value, N = Number, R = Return type.

Q: Can a static method use the class's type parameter?

No. Static methods need their own type parameter: `public static <U> U method(U input)`.

7.3 Bounded Type Parameters

```
// Upper bound
public <T extends Number> double sum(List<T> list) {
    double total = 0;
    for (T n : list) total += n.doubleValue();
    return total;
}

// Multiple bounds (one class first, then interfaces)
public <T extends Number & Comparable<T>> T max(List<T> list) { ... }
```

Type parameters can ONLY have upper bounds (`extends`), never lower bounds. `super` only works in **wildcards**.

7.4 Wildcards (?)

Wildcard	Meaning	Use case
<code><?></code>	Unbounded — any type	When type doesn't matter
<code><? extends T></code>	T or its subtypes	Read from collection (Producer)
<code><? super T></code>	T or its supertypes	Write to collection (Consumer)

7.5 PECS — Producer Extends, Consumer Super

If a collection **PRODUCES** values (you read) → use `<? extends T>`. If it **CONSUMES** values (you write) → use `<? super T>`. If both → plain `<T>`.

```

public static <T> void copy(List<? super T> dest, List<? extends T> src) {
    //           ^ consumer (write)           ^ producer (read)
    for (T t : src) dest.add(t);
}

List<Number> nums = new ArrayList<>();
List<Integer> ints = List.of(1, 2, 3);
copy(nums, ints); // works because Number super Integer

```

Q: Why can't you add to List<? extends Number>?

Compiler doesn't know exact type. Could be List<Integer> — adding Double would corrupt it.

Q: Why can't you read as Integer from List<? super Integer>?

Could be List<Object> — reading might return non-Integer. Can only read as Object.

7.6 Type Erasure

At **runtime**, generic type info is **erased** — replaced with raw types or upper bounds. Generics are purely a compile-time feature for backward compatibility with pre-Java 5 code.

```

// Source: Box<T> { T value; T get() { return value; } }
// Erased: Box    { Object value; Object get() { return value; } }

```

Consequences

1. `obj instanceof List<String>` — ✗ compile error (use `List<?>`)
2. Cannot create generic arrays: `new T[10]` — ✗
3. Cannot use type parameter in static context
4. Cannot create instance of T: `new T()` — ✗ (workaround: pass `Class<T>`)
5. Cannot overload methods with same erasure: `m(List<String>)` vs `m(List<Integer>)` — ✗
6. Cannot catch generic exceptions

7.7 Common Pitfalls

Generics are invariant

```
List<Object> objs = new ArrayList<String>(); // ✗ compile error
```

`List<String>` is NOT a subtype of `List<Object>`, even though String IS-A Object.

Arrays vs Generics

Arrays	Generics
Reified (type at runtime)	Erased
Covariant: <code>Object[] = String[]</code> ✓	Invariant: <code>List<Object> = List<String></code> ✗

Rule: Prefer Lists over arrays in generic code — generics catch errors at compile time.

7.8 Top Q&A

Q: What does PECS mean?

Producer Extends, Consumer Super. Use `<? extends T>` for read-only sources; `<? super T>` for write-only destinations.

Q: Difference between List<Object> and List<?>?

`List<Object>` = list of Objects only (invariant). `List<?>` = list of unknown type (can pass `List<String>` etc.).

Q: Can primitives be type parameters?

No. Use wrappers: `List<Integer>`, not `List<int>`.

Q: Difference between <T> and <?>?

`<T>` = named type parameter; reference it elsewhere. `<?>` = unnamed wildcard; use when you don't need to reference the type.

Topic 8 — Multithreading & Concurrency

8.1 Process vs Thread

Aspect	Process	Thread
Memory	Own memory space	Shares memory with other threads
Creation cost	Heavy	Light
Communication	IPC (slow)	Shared memory (fast)

8.2 Creating Threads (4 ways)

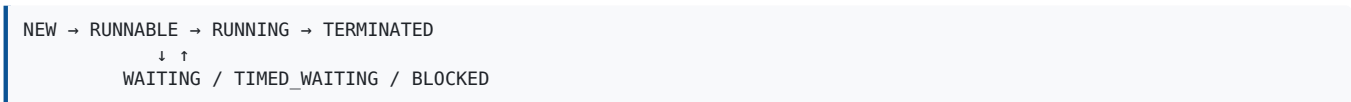
```
// 1. Extend Thread
class MyThread extends Thread { public void run() { ... } }
new MyThread().start();

// 2. Implement Runnable (preferred)
new Thread(() -> System.out.println("Lambda thread")).start();

// 3. Callable + ExecutorService (returns value, can throw)
ExecutorService exec = Executors.newSingleThreadExecutor();
Future<Integer> f = exec.submit(() -> 42);
Integer val = f.get();
exec.shutdown();
```

Always prefer Runnable/Callable over extending Thread: Java has single inheritance; Runnable separates task from thread; works with thread pools; Callable returns values and throws checked exceptions.

8.3 Thread Lifecycle



State	Meaning
NEW	Thread created, start() not called
RUNNABLE	Either running or ready to run
BLOCKED	Waiting to acquire synchronized lock
WAITING	wait(), join(), park() — indefinite
TIMED_WAITING	sleep(t), wait(t), join(t)
TERMINATED	run() finished

8.4 Multithreading & Race Condition

Multithreading = concurrent execution of two or more parts of a program for max CPU utilization.
Race condition = multiple threads access shared data, and the outcome depends on order of execution.

```
class Counter { int count = 0; public void increment() { count++; } }
// count++ is NOT atomic – 3 ops: read, add 1, write
// Two threads can interleave → lost updates
```

Problems caused by race conditions

- 1. Data corruption (lost updates, dirty reads)
- 2. Inconsistent state (invariants violated)
- 3. Non-deterministic bugs (works in dev, fails in prod)
- 4. Security vulnerabilities (TOCTOU)
- 5. Deadlocks / Livelocks
- 6. Incorrect aggregations (counters, sums)

How to prevent

- `synchronized` keyword
- Locks (`ReentrantLock`)
- Atomic classes
- Immutability
- Concurrent collections

8.5 synchronized Keyword

Ensures only one thread at a time can execute a block on a given monitor (lock). Provides **mutual exclusion + visibility** (happens-before).

4 ways to use

```
// 1. Synchronized instance method – locks 'this'
public synchronized void increment() { count++; }

// 2. Synchronized static method – locks Class object
public static synchronized void log() { }

// 3. Synchronized block – explicit lock object (preferred)
private final Object lock = new Object();
public void method() { synchronized(lock) { /* critical */ } }

// 4. Synchronized block on this
public void method() { synchronized(this) { } }
```

Counter example

```
class Counter {
    private int count = 0;
    public synchronized void increment() { count++; }
    public synchronized int getCount() { return count; }
}

Counter c = new Counter();
Thread t1 = new Thread(() -> { for (int i = 0; i < 10000; i++) c.increment(); });
Thread t2 = new Thread(() -> { for (int i = 0; i < 10000; i++) c.increment(); });
t1.start(); t2.start();
t1.join(); t2.join();
System.out.println(c.getCount()); // always 20000 ✓
```

Reentrancy: A thread holding a lock can re-enter the same lock without deadlock. Java's intrinsic locks are reentrant.

Q: Can a constructor be synchronized?

No — only one thread can construct a given object (no shared reference yet). Compile error.

Q: Two synchronized methods, one static and one instance — can two threads enter simultaneously?

Yes! Static locks `ClassName.class`, instance locks `'this'` — different monitors.

8.6 Thread.sleep()

Pauses the current thread for given milliseconds (static method). Throws **checked** `InterruptedException`. **Doesn't release locks.**

```
try {
    Thread.sleep(2000);
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
}
```

sleep() vs wait()

Feature	Thread.sleep()	Object.wait()
Class	Thread (static)	Object (instance)
Releases lock?	✗ No	✓ Yes
Where used	Anywhere	Only inside synchronized
Wakes via	Time / interrupt	notify / notifyAll / time / interrupt

8.7 wait(), notify(), notifyAll()

Methods of Object (not Thread). Must be called inside `synchronized` block on the same lock object.

Always wait inside a **loop** (spurious wakeups possible): `while (!condition) lock.wait();`

```
// Producer-Consumer
class Buffer {
    private final List<Integer> q = new ArrayList<>();
    private final int CAPACITY = 5;
    public synchronized void produce(int x) throws InterruptedException {
        while (q.size() == CAPACITY) wait();
        q.add(x); notifyAll();
    }
    public synchronized int consume() throws InterruptedException {
        while (q.isEmpty()) wait();
        int x = q.remove(0); notifyAll();
        return x;
    }
}
```

8.8 volatile Keyword

Ensures **visibility** of changes to a variable across threads. Reads always fetch from main memory; writes go straight to main memory. Prevents reordering.

Feature	volatile	synchronized
Visibility	✓	✓
Atomicity	✗	✓
Mutual exclusion	✗	✓
Blocking	✗	✓

```
class Worker implements Runnable {
    private volatile boolean running = true;
    public void run() { while (running) { /* work */ } }
    public void stop() { running = false; }
}
```

Double-checked locking with volatile

```
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized(Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

8.9 Atomic Classes

Lock-free, thread-safe operations on single variables using **CAS (Compare-And-Swap)**.

Common classes

Class	Type
AtomicInteger	int
AtomicLong	long
AtomicBoolean	boolean
AtomicReference<V>	object reference
LongAdder, LongAccumulator	high-contention counters (Java 8)

```
AtomicInteger counter = new AtomicInteger(0);
counter.incrementAndGet(); // ++count returns new
counter.getAndIncrement(); // count++ returns old
counter.compareAndSet(5, 10); // if==5, set to 10
counter.updateAndGet(x -> x * 2); // applies function
```

How CAS works

```
do {
    oldValue = get();
    newValue = oldValue + 1;
} while (!compareAndSet(oldValue, newValue));
```

Atomic vs synchronized

- Single variable, simple update → **Atomic** (faster, lock-free)
- Multiple variables, compound logic → **synchronized**
- Very high contention → **LongAdder** (better than AtomicLong)

8.10 Executor Framework

```
ExecutorService exec = Executors.newFixedThreadPool(4);

exec.execute(() -> System.out.println("task")); // Runnable
Future<Integer> f = exec.submit(() -> 42); // Callable
Integer val = f.get();

exec.shutdown();
exec.awaitTermination(5, TimeUnit.SECONDS);
```

Common pools

Method	Description
<code>newFixedThreadPool(N)</code>	Fixed N threads
<code>newCachedThreadPool()</code>	Grows/shrinks as needed
<code>newSingleThreadExecutor()</code>	Single worker
<code>newScheduledThreadPool(N)</code>	For scheduled tasks
<code>newWorkStealingPool()</code>	ForkJoinPool

ThreadPoolExecutor parameters

```
new ThreadPoolExecutor(
    int corePoolSize,
    int maximumPoolSize,
    long keepAliveTime, TimeUnit unit,
    BlockingQueue<Runnable> workQueue,
    RejectedExecutionHandler handler);
```

8.11 Future & CompletableFuture

Future (Java 5): Pending result. Has `get()` (blocking), `cancel()`, `isDone()`. No callbacks/chaining.

CompletableFuture (Java 8): Composition, callbacks, chaining.

```
CompletableFuture.supplyAsync(() -> "Hello")
    .thenApply(s -> s + " World")
    .thenApply(String::toUpperCase)
    .thenAccept(System.out::println); // "HELLO WORLD"

CompletableFuture<Integer> a = CompletableFuture.supplyAsync(() -> 10);
CompletableFuture<Integer> b = CompletableFuture.supplyAsync(() -> 20);
CompletableFuture<Integer> sum = a.thenCombine(b, Integer::sum);

CompletableFuture.allOf(a, b).join(); // wait for all
```

8.12 Lock Interface

```

Lock lock = new ReentrantLock();
lock.lock();
try {
    // critical section
} finally { lock.unlock(); } // ALWAYS in finally

```

Capabilities over synchronized

- `tryLock()` — non-blocking attempt
- `tryLock(timeout)` — wait limited time
- `lockInterruptibly()` — interruptible
- Fairness via `new ReentrantLock(true)`
- Multiple Conditions per lock

ReentrantReadWriteLock: multiple readers OR single writer — better for read-heavy.

8.13 Concurrency Utilities

Utility	Use case
CountDownLatch	Wait for N events (one-shot)
CyclicBarrier	All threads wait until N reach (reusable)
Semaphore	Limit concurrent access to N permits
Phaser	Multi-phase barrier
Exchanger	Swap data between two threads

8.14 Common Concurrency Problems

- **Deadlock** — Two+ threads waiting for each other's locks. Prevent: consistent lock ordering, tryLock with timeout.
- **Livelock** — Threads keep responding but no progress.
- **Starvation** — A thread can't get CPU/lock. Fix: fair locks.
- **Race condition** — Already covered.

8.15 Java Memory Model — Happens-Before

JMM defines visibility & ordering guarantees:

- synchronized exit → next acquire of same lock
- volatile write → subsequent read
- Thread.start() → first action of new thread
- All actions in thread → another thread's join() returning
- final field write in constructor → safe publication

8.16 ThreadLocal

Per-thread storage — each thread has its own copy.

```

ThreadLocal<SimpleDateFormat> df = ThreadLocal.withInitial(() -> new SimpleDateFormat("yyyy-MM-dd"));
String s = df.get().format(new Date());
df.remove(); // important – avoid memory leaks in pools

```

8.17 Top Q&A

Q: start() vs run()?

start() creates new thread, calls run() in it. Calling run() directly executes in current thread — no parallelism.

Q: Can we call start() twice?

No — IllegalStateException. Threads are one-shot.

Q: Why prefer Executor over new Thread()?

Thread reuse, bounded resources, task queueing, result handling (Future), lifecycle control.

Q: Why must wait/notify be inside synchronized?

They operate on the object's monitor. Otherwise → `IllegalMonitorStateException`.

Q: Why use while instead of if around wait()?

Spurious wakeups — JVM may wake without notify. Re-check the condition.

Q: Virtual threads (Java 21)?

Lightweight threads managed by JVM (not OS). Massively scalable for I/O-bound workloads.

Topic 9 — Java 8 Features

9.1 What Java 8 Brought

Feature	Purpose
Lambda Expressions	Concise anonymous functions
Functional Interfaces	Single-method interfaces (target for lambdas)
Stream API	Functional-style ops on collections
Method References	Shortcut for lambdas
Default & Static methods in interfaces	Evolve interfaces
Optional<T>	Null-safe container
java.time API	Modern Date/Time
CompletableFuture	Async programming

9.2 Lambda Expressions

```
// Syntax: (parameters) -> expression OR { statements }

Runnable r = () -> System.out.println("Hello");
Consumer<String> c = s -> System.out.println(s);
BiFunction<Integer, Integer, Integer> add = (a, b) -> a + b;
Comparator<Integer> cmp = (a, b) -> {
    if (a < b) return -1;
    if (a > b) return 1;
    return 0;
};
```

Lambda vs Anonymous Class

Feature	Lambda	Anonymous Class
Syntax	Concise	Verbose
<code>this</code>	Refers to enclosing class	Refers to anonymous class itself
Compiled to	Method (via invokedynamic)	New .class file
Can extend class?	✗ Only functional interfaces	✓ Any class/interface
Can have state?	✗	✓

Variable capture

Lambdas can use enclosing variables only if they're `final` or **effectively final**.

9.3 Functional Interfaces

Interface with exactly one abstract method (SAM). Optional `@FunctionalInterface` annotation.

Interface	Method	Description
Function<T,R>	R apply(T)	$T \rightarrow R$
BiFunction<T,U,R>	R apply(T,U)	$T, U \rightarrow R$
Predicate<T>	boolean test(T)	Boolean condition
Consumer<T>	void accept(T)	Takes T, no return
Supplier<T>	T get()	No input, returns T
UnaryOperator<T>	T apply(T)	$T \rightarrow T$
BinaryOperator<T>	T apply(T,T)	$T, T \rightarrow T$

Pre-Java 8 functional interfaces: Runnable, Callable, Comparator, Comparable.

Composing

```
Predicate<Integer> isEven = n -> n % 2 == 0;
Predicate<Integer> isPositive = n -> n > 0;
isEven.and(isPositive).test(4);      // true
isEven.or(isPositive).test(-3);     // true
isEven.negate().test(3);             // true

Function<Integer, Integer> times2 = n -> n * 2;
Function<Integer, Integer> plus3 = n -> n + 3;
times2.andThen(plus3).apply(5);     // (5*2)+3 = 13
times2.compose(plus3).apply(5);    // (5+3)*2 = 16
```

9.4 Method References

Type	Syntax	Lambda equivalent
Static method	ClassName::staticMethod	(args) -> ClassName.method(args)
Instance (specific obj)	obj::instanceMethod	(args) -> obj.method(args)
Instance (arbitrary obj)	ClassName::instanceMethod	(obj, args) -> obj.method(args)
Constructor	ClassName::new	(args) -> new ClassName(args)

```
Function<String, Integer> parse = Integer::parseInt;
Function<String, Integer> length = String::length;
Supplier<List<String>> listMaker = ArrayList::new;
strs.forEach(System.out::println);
```

9.5 Stream API

Pipeline of operations on a sequence of elements. **Streams don't store data** — they process from a source. Operations are **lazy** until terminal op runs. **Single-use** only.

Creating streams

```
list.stream(); list.parallelStream();
Arrays.stream(arr);
Stream.of(1, 2, 3);
Stream.empty();
Stream.iterate(0, n -> n + 1).limit(10);
Stream.generate(Math::random).limit(5);
IntStream.range(1, 10);           // 1..9
IntStream.rangeClosed(1, 10);    // 1..10
Files.lines(Paths.get("file.txt"));
```

Common Stream Methods

Intermediate operations (return Stream, lazy):

Method	Purpose
filter(Predicate)	Keep matching elements
map(Function)	Transform each element
flatMap(Function)	Flatten nested streams
sorted() / sorted(Comparator)	Sort
distinct()	Remove duplicates
limit(n) / skip(n)	Take first n / skip first n
peek(Consumer)	Debug — action without modifying
mapToInt / mapToLong / mapToDouble	Specialize to primitive stream

Terminal operations (eager, trigger execution):

Method	Purpose
forEach(Consumer)	Action on each
collect(Collector)	Accumulate into collection/map
toList() (Java 16+)	To unmodifiable List
count()	Number of elements
reduce(...)	Combine into single value
min/max(Comparator)	Returns Optional
findFirst / findAny	Returns Optional
anyMatch / allMatch / noneMatch	Boolean checks
toArray()	Array conversion
sum / average (on IntStream)	Numeric reductions

```
List<String> result = names.stream()
    .filter(n -> n.length() > 3)
    .map(String::toUpperCase)
    .sorted()
    .distinct()
    .limit(3)
    .collect(Collectors.toList());
```

9.6 Collectors

Collector	Purpose
toList(), toSet(), toMap(k, v)	To collection / map
joining(", ", "[", "]")	Concatenate strings
counting()	Count
summingInt(fn) / averagingDouble(fn)	Numeric aggregation
minBy(comp) / maxBy(comp)	Min/Max
groupingBy(classifier)	Group → Map<K, List<T>>
partitioningBy(predicate)	Split → Map<Boolean, List<T>>

```
Map<Integer, String> idToName = emps.stream()
    .collect(Collectors.toMap(Employee::getId, Employee::getName));

Map<String, Long> countByDept = emps.stream()
    .collect(Collectors.groupingBy(Employee::getDept, Collectors.counting()));

Map<String, Double> totalSalaryByDept = emps.stream()
    .collect(Collectors.groupingBy(Employee::getDept,
        Collectors.summingDouble(Employee::getSalary)));
```

toMap duplicate key handling

```
// Default – IllegalStateException on duplicate
Collectors.toMap(keyFn, valFn);

// Provide merge function
Collectors.toMap(keyFn, valFn, (existing, dup) -> existing);

// With map type
Collectors.toMap(keyFn, valFn, (a,b) -> a, LinkedHashMap::new);
```

9.7 Top 3 Users by Transaction Amount (real interview Q!)

```
// Setup
class Transaction { String userId; double amount; /* getters */ }

// Top 3 individual transactions
List<Transaction> top3 = transactions.stream()
    .sorted(Comparator.comparingDouble(Transaction::getAmount).reversed())
    .limit(3)
    .collect(Collectors.toList());

// Top 3 users by TOTAL transaction amount
List<Map.Entry<String, Double>> top3Users = transactions.stream()
    .collect(Collectors.groupingBy(
        Transaction::getUserId,
        Collectors.summingDouble(Transaction::getAmount)))
    .entrySet().stream()
    .sorted(Map.Entry.<String, Double>comparingByValue().reversed())
    .limit(3)
    .collect(Collectors.toList());

// Top 3 users by transaction COUNT (preserve order with LinkedHashMap)
Map<String, Long> top3 = transactions.stream()
    .collect(Collectors.groupingBy(Transaction::getUserId, Collectors.counting()))
    .entrySet().stream()
    .sorted(Map.Entry.<String, Long>comparingByValue().reversed())
    .limit(3)
    .collect(Collectors.toMap(
        Map.Entry::getKey, Map.Entry::getValue,
        (a, b) -> a, LinkedHashMap::new));
```

Trap: `Collectors.toMap()` returns regular `HashMap` → insertion order is lost! Use `LinkedHashMap::new` to preserve sorted order.

9.8 reduce()

```
// With identity (always returns T)
int sum = Stream.of(1,2,3,4).reduce(0, Integer::sum);    // 10

// Without identity (returns Optional)
Optional<Integer> max = Stream.of(1,2,3,4).reduce(Integer::max);

// With identity, accumulator, combiner (parallel-safe)
int totalChars = words.stream()
    .reduce(0, (acc, s) -> acc + s.length(), Integer::sum);
```

9.9 Parallel Streams

Uses `ForkJoinPool.commonPool()` — splits via `Splitter`, processes in parallel, merges.

When parallel helps

- Large datasets (typically > 10k elements)
- Independent operations
- CPU-intensive transformations

When parallel HURTS

- Small datasets (overhead exceeds benefit)
- I/O-bound operations
- Operations with side effects
- Order-dependent operations
- Boxed primitives (use `IntStream`)

9.10 Optional<T>

```
Optional<String> empty = Optional.empty();
Optional<String> of    = Optional.of("hi");           // throws NPE if null
Optional<String> ofN   = Optional.ofNullable(null);  // safe – empty if null

opt.isPresent(); opt.isEmpty();
opt.get();                               // throws if empty – avoid!
opt.orElse(defaultUser);                  // always evaluated
opt.orElseGet(() -> fetchDefault());       // lazy
opt.orElseThrow(() -> new MyException("..."));
opt.ifPresent(u -> log.info("Found: " + u));
opt.map(User::getName);
opt.flatMap(u -> Optional.ofNullable(u.getEmail()));
opt.filter(u -> u.getAge() > 18);
```

Q: orElse vs orElseGet?

`orElse(val)` always evaluates `val`. `orElseGet(supplier)` only runs supplier if Optional is empty. Use `orElseGet` for expensive defaults.

9.11 Default & Static Methods in Interfaces

```
interface Logger {
    default void info(String msg) { log("INFO", msg); }
    default void warn(String msg) { log("WARN", msg); }
    private void log(String level, String msg) {           // Java 9+
        System.out.println(level + ": " + msg);
    }
    static Logger getDefault() { return new ConsoleLogger(); }
}
```

9.12 New Date & Time API (java.time)

Class	Represents
LocalDate	Date without time
LocalTime	Time without date
LocalDateTime	Date + time (no zone)
ZonedDateTime	Date + time + timezone
Instant	Machine timestamp
Duration	Time-based amount
Period	Date-based amount

```
LocalDate today = LocalDate.now();
LocalDate dob = LocalDate.of(1995, 6, 15);
LocalDate tomorrow = today.plusDays(1);

Period age = Period.between(dob, today);
Duration d = Duration.between(start, end);

DateTimeFormatter fmt = DateTimeFormatter.ofPattern("dd-MM-yyyy");
String str = today.format(fmt);
LocalDate parsed = LocalDate.parse("15-03-2024", fmt);
```

9.13 Useful Map & Collection Additions

```
list.forEach(System.out::println);
list.removeIf(s -> s.startsWith("a"));

Map<String, Integer> wc = new HashMap<>();
wc.merge("hello", 1, Integer::sum);
wc.computeIfAbsent("key", k -> new ArrayList<>());
wc.computeIfPresent("key", (k, v) -> v * 2);
int count = wc.getOrDefault("missing", 0);
```

9.14 Top Q&A

Q: map() vs flatMap()?

`map(T -> R) : Stream<R>`. `flatMap(T -> Stream<R>) : flattens nested → Stream<R>`.

Q: Can a stream be reused?

No. After terminal op runs, stream is closed. Reuse → `IllegalStateException`.

Q: findFirst() vs findAny()?

`findFirst`: first in encounter order. `findAny`: any element (faster on parallel streams).

Q: What is short-circuiting?

Operations that may not process all elements: `findFirst`, `findAny`, `anyMatch`, `allMatch`, `noneMatch`, `limit`. Terminate early.

Q: Collectors.toList() vs Stream.toList()?

`Collectors.toList()` → `ArrayList` (mutable). `Stream.toList()` (Java 16+) → unmodifiable `List`.

Topic 10 — Memory Management & Garbage Collection

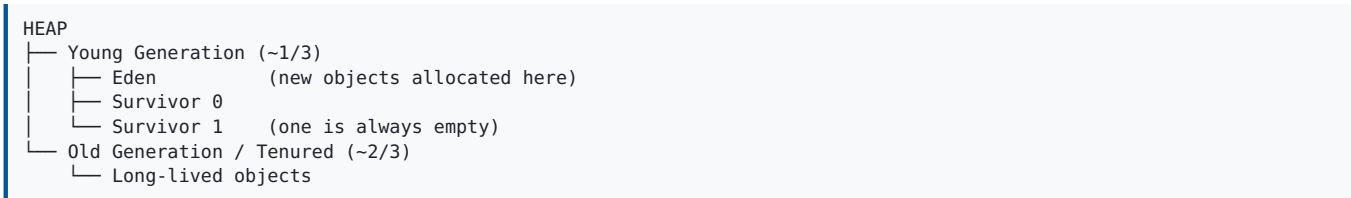
10.1 JVM Memory Architecture

Region	Stored	Shared?	GC'd?
Heap	Objects, arrays, instance variables	✓ All threads	✓ Yes
Stack	Method frames, local vars, references	✗ Per thread	✗ Auto (LIFO)
Metaspace	Class metadata, methods, static vars	✓	✓ (class unloading)
PC Register	Current instruction address	✗ Per thread	—
Native Method Stack	JNI frames	✗ Per thread	—
Code Cache	JIT-compiled native code	✓	—

Heap vs Stack

Aspect	Heap	Stack
Stores	Objects	Primitives, references, frames
Shared	Yes	No (per thread)
Size	Larger (GBs)	Smaller (KB-MB per thread)
Allocation	new keyword	Method invocation
Deallocation	GC	Auto on method return
Errors	OutOfMemoryError: Java heap space	StackOverflowError
Flags	-Xms / -Xmx	-Xss

10.2 Heap Structure (Generational)



Weak Generational Hypothesis: most objects die young → concentrate GC effort on young gen.

Object lifecycle

- Object created → Eden
- Eden full → **Minor GC**: live objects copied to Survivor (S0/S1 toggle); dead discarded
- Survivor age increments each minor GC
- Once age > tenuring threshold (default 15) → promoted to Old Gen
- Old Gen full → **Major / Full GC** (slower)

GC types

GC Type	Scope	Speed	Frequency
Minor GC	Young Gen only	Fast	Frequent
Major GC	Old Gen	Slow	Less frequent
Full GC	Entire heap + Metaspace	Slowest	Rare (and bad)

10.3 What is Garbage Collection?

Automatic memory management — JVM reclaims memory of objects no longer reachable from **GC Roots**.

GC Roots

1. Local variables in active methods (stack frames)
2. Static variables of loaded classes
3. Active threads
4. JNI references (native code)
5. Synchronization monitors held by threads

Object reachability

State	Meaning
Strongly reachable	Through chain from GC Root → never GC'd
Softly reachable	Only via SoftReference → GC'd before OOM
Weakly reachable	Only via WeakReference → GC'd at next cycle
Phantomly reachable	Already finalized; cleanup pending
Unreachable	Eligible for GC

```
Object a = new Object();    // a → obj1
Object b = new Object();    // b → obj2
a = b;                     // obj1 unreachable → eligible for GC
b = null;                   // obj2 still reachable via 'a'
a = null;                   // now obj2 unreachable too
```

10.4 GC Algorithms

1. **Mark and Sweep** — mark reachable, sweep unmarked. Causes fragmentation.
2. **Mark-Compact** — adds compaction. Eliminates fragmentation; slower.
3. **Copying / Mark-Copy** — splits heap, copies live to "to" space. Used in Young Gen.
4. **Generational** — different algorithms for Young vs Old. All modern JVMs.
5. **Concurrent / Parallel** — multiple GC threads; some phases run with app threads.

10.5 Garbage Collectors

Collector	Default in	Best for
Serial GC	Small apps, single-CPU	Embedded, low memory
Parallel GC	Java 8 default (server)	Throughput-focused batch jobs
CMS	Java 4-8 (deprecated, removed Java 14)	Low pause
G1 GC	Java 9+ default	Balanced
ZGC	Java 11+	Ultra-low pause, large heaps
Shenandoah	Java 12+	Low pause

G1 GC (default since Java 9)

- Heap divided into regions (~1MB-32MB each)
- Pause time goal via `-XX:MaxGCPauseMillis`
- Prioritizes regions with most garbage first → "Garbage First"
- Concurrent marking + parallel collection

Common JVM flags

```
-Xms512m          # initial heap
-Xmx2g            # max heap
-Xss256k          # thread stack size
-XX:+UseG1GC      # use G1
-XX:+UseZGC       # use ZGC
-XX:MaxGCPauseMillis=200 # G1 pause target
-XX:+HeapDumpOnOutOfMemoryError # dump heap on OOM
-Xlog:gc*         # GC logs (Java 9+)
```

10.6 Reference Types

Type	When GC'd	Use case
Strong (default)	Never while reachable	Normal references
Soft	Before OOM	Memory-sensitive caches
Weak	Next GC cycle	Auxiliary maps (WeakHashMap)
Phantom	After finalization	Cleanup tracking

10.7 finalize() and modern alternatives

`finalize()` — DEPRECATED since Java 9, removed Java 18. Unreliable, slow, allows resurrection.

Modern alternatives:

```
// 1. AutoCloseable + try-with-resources (preferred)
try (Resource r = new Resource()) { /* use */ }

// 2. Cleaner (Java 9+)
private static final Cleaner cleaner = Cleaner.create();
cleaner.register(this, () -> { /* cleanup */ });
```

10.8 Memory Leaks in Java

Java has GC, but leaks still occur when objects remain reachable but no longer needed.

Common causes

1. **Static collections that grow unbounded** — adds keep happening, never removed
2. **Forgotten listeners/callbacks** — never deregistered
3. **Inner classes / lambdas** holding outer references
4. **ThreadLocal in thread pools** — values accumulate; always `remove()` in finally
5. **ClassLoader leaks** — common in app servers (Tomcat undeploys)
6. **String pool / interning** abuse
7. **Unclosed resources** (Connection, Stream, Statement)
8. **Mutable keys in HashMap** — hashCode changes after insertion

10.9 OutOfMemoryError Variants

Error	Cause	Fix
Java heap space	Heap full	Increase -Xmx, fix leaks
GC overhead limit exceeded	GC takes >98% time, recovers <2%	Same as above
Metaspace	Too many loaded classes	-XX:MaxMetaspaceSize, classloader leaks
unable to create new native thread	OS thread limit	Reduce thread count
Direct buffer memory	NIO direct buffers exhausted	-XX:MaxDirectMemorySize

StackOverflowError = stack frame depth exceeded (usually infinite recursion). Different from OOM. Increase via `-Xss` or fix recursion.

10.10 Tools to Diagnose Memory Issues

Tool	Purpose
jps	List Java processes
jstat	Real-time GC statistics
jmap	Generate heap dump, histogram
jhat / Eclipse MAT	Analyze heap dumps
jstack	Thread dump
VisualVM	Visual profiling
JFR	Built-in production profiling
JConsole	JMX monitoring

10.11 System.gc() — should you call it?

It's only a **hint**. JVM may ignore. Triggers Full GC if it runs — expensive. **Avoid**. Can be disabled via `-XX:+DisableExplicitGC`.

10.12 Best Practices

1. Minimize object creation in tight loops
2. Use primitives over wrappers in hot code
3. Pre-size collections
4. Close resources (try-with-resources)
5. Use weak/soft references for caches
6. Beware of static collections — they live forever
7. Avoid `String.intern()` for unbounded user input
8. Profile before optimizing

10.13 Top Q&A

Q: Difference between heap and stack?

Stack: per-thread, method frames + locals, fast, auto-cleared. Heap: shared, objects, GC-managed, larger.

Q: When is an object eligible for GC?

When no live thread can reach it via any chain of references from GC Roots.

Q: Why generational heap?

Most objects die young — concentrating GC on young gen is faster. Weak Generational Hypothesis.

Q: Stop-the-World?

Pause where ALL app threads suspend so GC can work safely. Even concurrent GCs have brief STW phases.

Q: Default GC in Java 8 vs 11?

Java 8 (server): Parallel GC. Java 9+: G1 GC.

Q: PermGen vs Metaspace?

PermGen: fixed size, in heap, prone to OOM. Metaspace (Java 8+): native memory, auto-grows, more flexible.

Q: Why doesn't Java use reference counting?

Reference counting can't reclaim cycles. Java uses tracing GC (mark-sweep variants) which handles cycles correctly.

Q: What is escape analysis?

JIT optimization — if an object never "escapes" its method, it can be allocated on stack or scalar-replaced → no heap allocation, no GC pressure.

Q: Island of Isolation?

Two+ objects reference each other (cycle) but nothing outside references any of them → all eligible for GC. Java's tracing GC handles cycles correctly (unlike reference counting).

End of Core Java Roadmap — All 10 Topics Complete
Next up: Advanced Java (JDBC, Spring, JPA, Testing, Design Patterns)